ACCESO A DATOS – 2º DAM

UP 2 – Bases de Datos Relacionales - JDBC

SESIÓN 2 - PERSISTENCIA DE DATOS, MODELO DE ENTIDADES (POJO), SENTENCIAS SQL Y PATRÓN DAO

OBJETIVO ACADÉMICO (RA 2 - CRITERIOS A, B, C, D)
Tras verificar la conexión a mutxamel_pg, en esta sesión implementaremos la **persistencia de datos** en el proyecto «Mutxamel Sec-Ops S.L.». Analizaremos la volatilidad de la memoria, mapearemos tablas de PostgreSQL a clases Java (**POJOs**), ejecutaremos consultas mediante la **API JDBC** y estructuraremos todo el código utilizando el **Patrón DAO.**

1. Persistencia de la Información: Memoria Volátil (Heap) frente a RDBMS

La **persistencia** es la capacidad de conservar los objetos en un soporte no volátil para que no desaparezcan al finalizar el proceso que los creó. Por defecto, cuando creamos un objeto en Java, este se aloja en el **Heap** (memoria dinámica de la JVM), un espacio volátil que el *Garbage Collector* limpia automáticamente al cerrar la aplicación.

Aunque en la UP1 utilizamos ficheros para guardar información, este mecanismo tiene serias limitaciones: carece de control de concurrencia, el rendimiento cae en búsquedas complejas y no garantiza la integridad referencial. Para superar estas barreras y gestionar grandes volúmenes de información, utilizamos los **Sistemas Gestores de Bases de Datos Relacionales (SGBDR)**, como PostgreSQL o MariaDB.

```java
// Estado transitorio en el Heap de Java frente a necesidad de persistencia relacional
public class User {
    private int id;
    private String name;
    private String email;
    private Timestamp creationDate;
    // Constructores, Getters y Setters...
}
// Al ejecutar esta instrucción, la instancia vive únicamente en la memoria RAM (Heap de la JVM):
Timestamp ts = new Timestamp(System.currentTimeMillis());
User user = new User(1, "Silvia", "silvia@silvia.es", ts);
```

La imagen muestra el problema de la memoria volátil:

- Definimos los atributos del objeto (`id, name, email, creationDate`).
- Al usar `new`, Java guarda los datos del usuario temporalmente en el Heap.
- Podemos operar con ellos a gran velocidad mientras el programa siga en ejecución.
- Al cerrar la aplicación, la información se destruye. Para evitar esta pérdida, es imprescindible persistir los datos en un SGBDR como PostgreSQL.

2. El Desfase Objeto-Relacional (Object-Relational Impedance Mismatch)

Al integrar Java con bases de datos relacionales —en nuestro caso, utilizaremos **PostgreSQL** de forma central a lo largo del curso— surge un choque conceptual conocido como **desfase objeto-relacional** (*Impedance Mismatch*). Este desajuste se debe a que ambos paradigmas operan bajo reglas estructurales completamente opuestas:

- **Estructura de Datos:** Java encapsula información y comportamiento en objetos, mientras que PostgreSQL almacena datos atómicos en tablas planas bidimensionales.
- **Identidad:** En memoria, los objetos se distinguen por su referencia o contenido, mientras que en base de datos requieren una Clave Primaria (PK) estricta.
- **Asociaciones:** Java navega entre objetos usando colecciones directas (como listas o sets); el modelo relacional exige Claves Foráneas (FK) y tablas intermedias.
- **Herencia:** Las jerarquías de clases son nativas en Java, pero no existen de forma natural en las tablas estándar.

La siguiente imagen ilustra gráficamente este problema. Mientras que en Java una relación muchos a muchos (N:M) se resuelve de forma directa mediante colecciones en memoria (Set<Proyecto>), en PostgreSQL esa estructura directa no es viable y nos obliga a descomponerla en tres tablas normalizadas conectadas por claves foráneas.



Figura 2.1. Transformación de un grafo bidireccional N:M en memoria a tres tablas normalizadas mediante claves primarias y foráneas.

3. Arquitectura de conectores: La especificación JDBC

Para conectar Java con PostgreSQL de forma independiente del fabricante, utilizamos la especificación **JDBC** (*Java Database Connectivity*) dentro del paquete `java.sql`. JDBC define interfaces comunes (Connection, Statement, ResultSet) que implementan los drivers específicos de cada motor. Para utilizarlo en nuestro proyecto, debemos descargar el driver añadiendo su artefacto correspondiente dentro de las dependencias del archivo `pom.xml` de Maven:

Motor SGBDR	Formato de URL de Conexión (Protocolo JDBC)	Artefacto Maven (GroupId : ArtifactId)
PostgreSQL	jdbc:postgresql://localhost:5432/mutxamel_pg	org.postgresql : postgresql (v42.7.4)
MariaDB	jdbc:mariadb://localhost:3306/libro_ad	org.mariadb.jdbc : mariadb-java-client (v3.5.3)
MySQL	jdbc:mysql://localhost:3306/empresa	com.mysql : mysql-connector-j (v8.4.x)
Oracle DB	jdbc:oracle:thin:@localhost:1521:xe	com.oracle.database.jdbc : ojdbc11

4. Establecimiento de la Conexión y Diagnóstico de SQLException

Para abrir la sesión con la base de datos utilizamos DriverManager.getConnection(). Al ser una operación costosa tanto en red como en recursos del SGBD, debemos envolverla siempre en un bloque **try-with-resources**. Así aseguramos su cierre automático de forma limpia y evitamos dejar conexiones abiertas o fugas de memoria en el servidor.

Clase auxiliar para diagnóstico de SQLException

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class JDBCConnectionDiagnostic {

    public static void muestraErrorSQL(SQLException e) {
        System.err.printf("Ups, hemos detectado un problema con la DB! %n");
        System.err.printf("¿Qué ha pasado?: %s%n", e.getMessage());
        System.err.printf("Código de estado (SQLState): %s%n", e.getSQLState());
        System.err.printf("Código interno del motor: %d%n", e.getErrorCode());

        SQLException next = e.getNextException();
        while (next != null) {
            System.err.printf(" -> Además,provocó este otro fallo: %s (Estado: %s)%n",
next.getMessage(), next.getSQLState());
            next = next.getNextException();
        }
    }

    public static void main(String[] args) {
        String urlPg = "jdbc:postgresql://localhost:5432/mutxamel_pg";

        // try-with-resources cierra la conexión automáticamente al implementar AutoCloseable
        try (Connection con = DriverManager.getConnection(urlPg, "postgres", "postgres")) {
            System.out.println("¡Genial! Hemos conectado a la base de datos con éxito.");
            System.out.printf("Estamos trabajando con: %s%n",
con.getMetaData().getDatabaseProductName());
        } catch (SQLException ex) {
            muestraErrorSQL(ex);
        }
    }
}
```

Anatomía del Diagnóstico (y su importancia en el proyecto)

En desarrollos reales como el de Mutxamel Sec-Ops, esta clase es vital porque nos permite aislar rápidamente si un fallo de conexión es culpa de nuestro código, de una caída de la red o de unas credenciales incorrectas. La clase imprime el error en la consola usando System.err.printf (%s para insertar textos, %d para números y %n para el salto de línea), desglosando la información en tres niveles críticos:

- **e.getMessage():** El texto descriptivo exacto que nos devuelve PostgreSQL (ej. "autenticación de contraseña fallida").

- **e.getSQLState():** Un código estándar internacional (ANSI/ISO). Permite identificar el fallo independientemente del motor de base de datos que usemos (ej. 28P01 siempre indica credenciales inválidas).
- **e.getErrorCode():** El código numérico interno del fabricante, esencial para buscar soluciones rápidas en la documentación oficial de PostgreSQL.

5. Reto práctico y validación presencial

Para afianzar lo aprendido, crea la clase JDBCConnectionDiagnostic dentro del paquete **ud2.util** de tu proyecto (agrupar estas herramientas genéricas aquí es una buena práctica profesional que reaprovecharemos más adelante).

Ejecútala para establecer primero una conexión exitosa con tu DB local mutxamel_pg. A continuación, fuerza un error intencionado en las credenciales (por ejemplo, introduciendo una contraseña incorrecta) y comprueba en la consola cómo el sistema captura y desglosa detalladamente el código de estado SQL.